

HeartCoin

Empresa: The Original Lab

Área responsable: Dirección General

Responsables del proyecto: Erik Rios · Aldo Herrera

Fecha de elaboración: 03 de agosto de 2026

Documento técnico oficial — Documentación de cierre de proyecto de estadías profesionales.

CONTROL DEL DOCUMENTO

Campo	Detalle
Versión del documento	1.1
Fecha de actualización	03/agosto/2026
Autores	Erik Rios, Aldo Herrera
Estado	Entrega final

Historial de cambios

Versión	Fecha	Autor(es)	Descripción del cambio
1.0	03/08/2026	Erik Rios, Aldo Herrera	Emisión de la versión final del documento técnico para entrega a The Original Lab
1.1	03/08/2026	Erik Rios, Aldo Herrera	Actualización de la sección 4 (Componentes principales) para reflejar la reorganización de <code>lib/screens/</code> , <code>lib/services/</code> y <code>lib/widgets/</code> en subcarpetas por rol de usuario

1. INTRODUCCIÓN

Contexto del proyecto

HeartCoin es una aplicación móvil desarrollada para Android e iOS con Flutter, respaldada por Supabase como backend (base de datos Postgres, autenticación, tiempo real y funciones serverless). El proyecto fue desarrollado en el marco del programa de estadías profesionales de los responsables Erik Rios y Aldo Herrera, bajo la dirección de The Original Lab, y se integra activamente con dos plataformas propias de la empresa: **Original Auth** (verificación de identidad) y **Original Pay** (gestión de saldo y cobros).

El desarrollo tuvo un origen previo en tecnología Expo/React Native (enero de 2026) que posteriormente fue reescrito íntegramente en Flutter. El grueso del trabajo documentado en este entregable corresponde a un periodo de desarrollo intensivo concentrado entre el 20 y el 31 de julio de 2026, precedido de trabajo iterativo por rol de usuario desde mayo de 2026.

Motivo de desarrollo

The Original Lab requería una aplicación propia que permitiera conectar en un mismo ecosistema económico a tres tipos de actores: personas que participan en causas sociales, organizaciones (fundaciones/ONGs) que impulsan esas causas, y empresas que ofrecen beneficios reales canjeables con una moneda de participación (HeartCoin / HC). El proyecto nace para resolver la falta de un canal digital propio que uniera "hacer el bien" con "recibir un beneficio tangible", aprovechando además la infraestructura de identidad y pagos ya construida por The Original Lab (Original Auth, Original Pay).

Importancia para la empresa

HeartCoin representa la primera aplicación de consumo que integra de punta a punta los servicios de Original Auth y Original Pay, sirviendo como caso de uso de referencia y validación funcional de ambas plataformas en un producto real, además de constituir en sí mismo un activo de producto para The Original Lab con marca propia (`com.theoriginallab.heartcoin`).

Alcance del documento

Este documento cubre la descripción funcional y técnica completa de HeartCoin: arquitectura, modelo de datos, integraciones externas, metodología y actividades de desarrollo, manual técnico de instalación/operación, resultados obtenidos durante el periodo de estadías, y recomendaciones de mantenimiento y evolución futura. Está dirigido a cualquier integrante del área técnica de The Original Lab que necesite comprender, operar, mantener o extender el sistema.

2. DESCRIPCIÓN GENERAL DEL PROYECTO

Objetivo principal

Desarrollar y poner en operación una aplicación móvil multiplataforma (Android/iOS) que permita a personas participar en iniciativas sociales y campañas de votación de organizaciones, obtener HeartCoin (HC) como moneda de participación, y canjear ese HC por beneficios y servicios reales ofrecidos por empresas — integrando para ello los servicios propios de identidad (Original Auth) y pagos (Original Pay) de The Original Lab.

Alcance

El alcance funcional cubierto por el proyecto incluye:

- Tres roles de usuario diferenciados: **Personal**, **Organización** y **Empresa**, cada uno con su propio flujo de registro, perfil y panel.
- Feed social con publicaciones, likes, comentarios (con respuestas), seguimiento entre usuarios/organizaciones y notificaciones.
- Iniciativas por categoría (Voluntariado, Crowdfunding, Social, Ahorro colectivo) con check-in por QR y geolocalización.
- Campañas de votación publicadas por organizaciones, con comentarios y otorgamiento de HC por participación.
- Billetera de HC con historial de transacciones (ledger único).
- Catálogo de beneficios y servicios publicados por empresas, canjeables con HC, georreferenciados en mapa.
- Certificados de participación, historial de acciones del usuario, elementos guardados, bloqueo de usuarios.
- Verificación de correo electrónico real en el registro mediante Original Auth.
- Consulta de saldo HC y registro de cobros con HC desde sistemas externos mediante Original Pay.
- Almacenamiento externo de imágenes y documentos mediante el servicio MediaAAS.

Usuarios o áreas involucradas

- **Personal** (usuario final): participa en iniciativas y votaciones, acumula y gasta HC.
- **Organización**: fundaciones, ONGs y asociaciones civiles que publican iniciativas y votaciones.
- **Empresa**: negocios que publican beneficios y servicios canjeables con HC.
- **Dirección General de The Original Lab**: área responsable del proyecto.
- **Equipo técnico de The Original Lab**: consumidor final de este documento para mantenimiento y evolución del sistema.

Beneficios esperados

- Un producto propio de The Original Lab que demuestra y valida en producción las capacidades de Original Auth y Original Pay.

- Un canal digital que conecta participación social con beneficios económicos reales, incentivando tanto a organizaciones como a empresas a sumarse al ecosistema.
- Una base de código y arquitectura (Flutter + Supabase) de bajo costo operativo, sin backend propio que mantener más allá de Postgres/Edge Functions.

3. SITUACIÓN INICIAL

Proceso o sistema existente antes del proyecto

No existía previamente una aplicación de HeartCoin operativa en producción. El proyecto tuvo un primer intento de implementación en Expo/React Native (commits de enero de 2026: `Initial commit`, `feat: bootstrap HeartCoin Expo app`), que no continuó como base del producto final.

Problemas identificados

- La primera base tecnológica (Expo/React Native) fue descartada y sustituida por una reescritura completa en Flutter, documentada en el commit `Initial Flutter frontend import from hearthy`.
- No existía integración entre un sistema de participación social/iniciativas y los servicios de identidad y pagos propios de The Original Lab.
- No había un canal donde empresas pudieran ofrecer beneficios reales a cambio de participación social verificable.

Limitaciones encontradas

- Durante el desarrollo se identificaron limitaciones en la integración con Original Auth, particularmente en el envío del campo `country` (formato ISO alpha-2 no coincide con el texto libre capturado en el formulario de registro), documentadas en `ORIGINAL_AUTH_INTEGRATION.md`.
- El ambiente de pruebas (`test`) de Original Auth no envía correos reales, entregando un `testToken` directo, lo cual debió considerarse en el diseño del flujo de verificación.
- El webhook de Original Pay no cuenta con firma criptográfica real (diseño del lado de Original Pay, ajeno al alcance de HeartCoin), lo que se documenta como consideración de seguridad a vigilar.

Necesidades de mejora

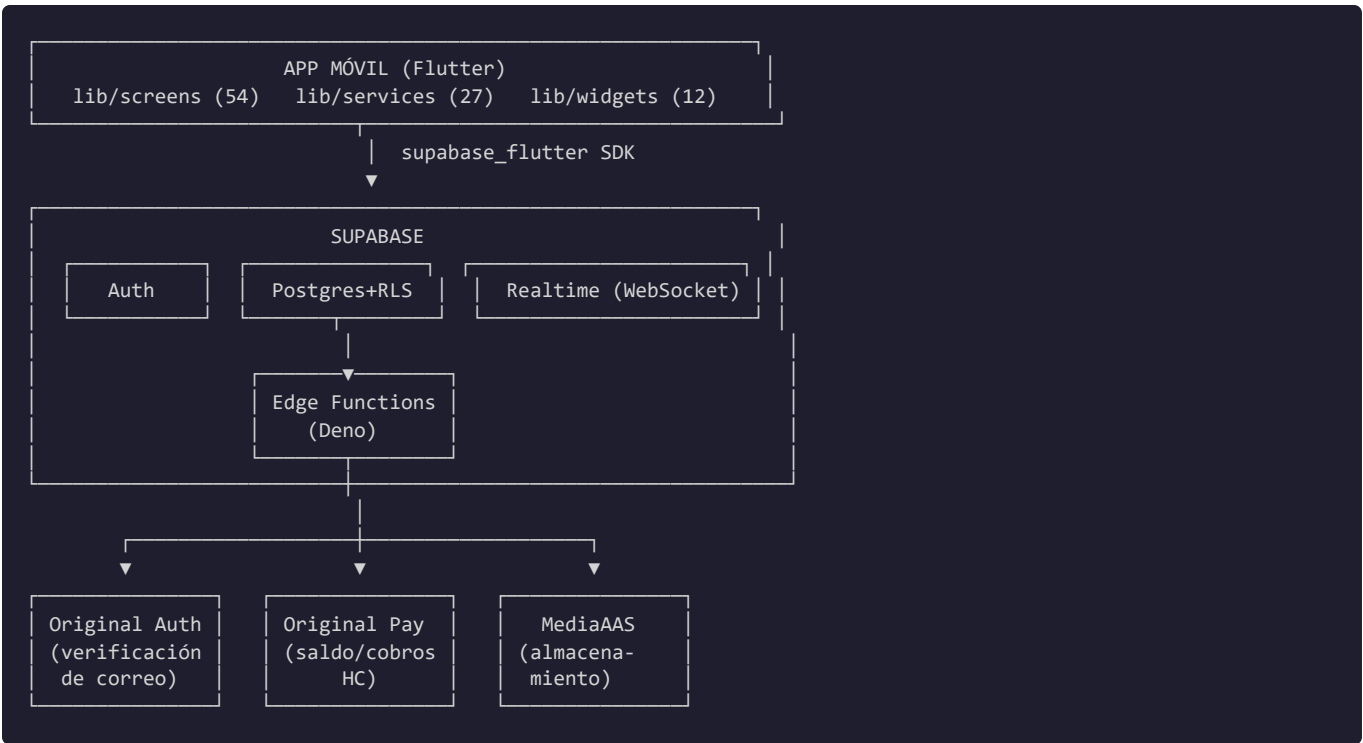
- Contar con un modelo de datos versionado mediante migraciones SQL (antes las modificaciones de esquema se aplicaban directamente en Supabase Studio, según se reconoce en `MANUAL_TECNICO.md`).
- Consolidar en un único documento la información dispersa en múltiples archivos Markdown y PDFs generados durante el desarrollo.

4. SOLUCIÓN IMPLEMENTADA

Descripción de la solución

HeartCoin se implementó como una aplicación Flutter (Android/iOS) sin backend propio: toda la lógica de negocio reside en Supabase, aprovechando Postgres con Row Level Security (RLS), funciones `SECURITY DEFINER` para operaciones sensibles, canales de tiempo real para actualizaciones en vivo, y Edge Functions (Deno) para la integración con servicios externos (Original Auth, Original Pay, MediaAAS).

Arquitectura general



Componentes principales

Frontend (Flutter — lib/):

El código fuente se reorganizó por rol de usuario, separando en subcarpetas los archivos comunes de los específicos de cada perfil (Personal, Organización, Empresa), además de aislar autenticación, onboarding y la pantalla raíz de inicio:

Carpeta	Contenido	Volumen
lib/main.dart	Rutas, tema, escalado de texto global, credenciales de Supabase	1 archivo
lib/screens/auth/	Login, registro, verificación de correo, recuperación/reseteo de contraseña	6 archivos
lib/screens/home/	Pantalla raíz de inicio (enrutamiento por rol)	1 archivo

Carpeta	Contenido	Volumen
<code>lib/screens/onboarding/</code>	Pantalla de onboarding inicial	1 archivo
<code>lib/screens/common/</code>	Pantallas compartidas entre roles: perfil, configuración, notificaciones, privacidad, bloqueados, acerca de, etc.	14 archivos
<code>lib/screens/personal/</code>	Pantallas exclusivas del rol Personal: feed, iniciativas, votaciones, billetera, certificados, escáneres QR	22 archivos
<code>lib/screens/organizacion/</code>	Pantallas exclusivas del rol Organización: comunidad, iniciativas propias, monitor de votaciones	8 archivos
<code>lib/screens/empresa/</code>	Pantallas exclusivas del rol Empresa: beneficios, servicios, QR de canje	7 archivos
<code>lib/services/common/</code>	Servicios compartidos: auth, cuentas, ubicación, seguir, bloqueos, notificaciones, Original Auth, media, guardados, escalado de texto, redes sociales	17 archivos
<code>lib/services/personal/</code>	Servicios exclusivos de Personal: check-in, wallet, ranking, certificados, participantes de iniciativa, comentarios de votación, mis acciones	7 archivos
<code>lib/services/organizacion/</code>	Servicios exclusivos de Organización: comunidad, dashboard de organización	2 archivos
<code>lib/services/empresa/</code>	Servicio exclusivo de Empresa: dashboard de empresa	1 archivo
<code>lib/widgets/common/</code>	Componentes reutilizables (tarjetas de iniciativas, botón seguir/guardar, grid de publicaciones, navegación inferior, componentes de mapa)	12 archivos
<code>lib/theme/app_colors.dart</code>	Paleta de color oficial de la app	1 archivo

Nota: la reorganización de `lib/screens/`, `lib/services/` y `lib/widgets/` en subcarpetas por rol es un cambio estructural reciente respecto a la organización plana descrita en versiones previas de la documentación del proyecto (README.md, MANUAL_TECNICO.md); no altera funcionalidad, solo la ubicación física de los archivos fuente.

Backend (Supabase, proyecto `heartcoin-v1`, ID `bamrvwzpcqwoyuwbvomw`):

- **Auth:** única fuente de verdad de identidad (registro, login, sesión, recuperación de contraseña).
- **Postgres + RLS:** ~20 tablas con seguridad a nivel de fila en todas ellas.
- **Realtime:** usado en `publicaciones`, `comments`, `iniciativas`, `votaciones`, `beneficios`, `servicios`, `hc_transactions`, `personal_profiles`, `votacion_comments`, `saved_items`.
- **Edge Functions (Deno):** `upload-media`, `delete-account`, `originalauth-register`, `originalauth-verify-email`, `originalauth-resend-verification`, `originalpay-balance`, `originalpay-redeem`, `originalpay-webhook`, y el cliente compartido `_shared/originalauth.ts`.

Flujo de funcionamiento (ejemplo — participación y canje de HC)

1. El usuario Personal se registra en la app → la cuenta se crea en Supabase Auth → HeartCoin llama a Original Auth para verificar el correo mediante un código de 6 dígitos.
2. El usuario participa en una iniciativa (voluntariado, crowdfunding, causa social o ahorro) haciendo check-in por QR con geolocalización, o participa en una votación de una organización.

3. La participación otorga HC, registrado como movimiento en la tabla `hc_transactions` (ledger único).
4. El usuario consulta beneficios/servicios publicados por empresas (georreferenciados en mapa) y canjea HC por ellos mediante las funciones `redeem_beneficio` / `redeem_servicio`.
5. Externamente, un sistema de Original Pay puede consultar el saldo HC de un usuario por correo (`originalpay-balance`) y registrar un cobro con HC (`originalpay-redeem`), recibiendo confirmación o rechazo vía `originalpay-webhook` (con reembolso automático en caso de rechazo).

Herramientas utilizadas

Ver tabla completa de dependencias en la sección 6 (Manual Técnico).

Configuraciones importantes

- **Application ID Android:** `com.theoriginallab.heartcoin` (anteriormente `com.example.heartcoin` , actualizado para publicación en Google Play, incluyendo generación de keystore de firma).
- **Base URL de Supabase:** `https://bamrvwzpcqwoyuwbvomw.supabase.co`.
- **Autenticación de mapas:** la app utiliza **Google Maps** mediante el paquete `google_maps_flutter` (confirmado en `pubspec.yaml`). *Nota: documentación previa del proyecto (README.md, MANUAL_TECNICO.md) hacía referencia a una implementación basada en OpenStreetMap/flutter_map; esa referencia quedó obsoleta y debe considerarse a Google Maps como la tecnología de mapas vigente.*

Integraciones realizadas

Integración	Función	Estado
Original Auth	Verificación de correo real en el registro	Implementado
Original Pay	Consulta de saldo HC y registro de cobros externos	Implementado y probado de punta a punta, incluyendo reembolso automático
MediaAAS	Almacenamiento externo de imágenes/documentos vía proxy Edge Function	Implementado

5. DESARROLLO E IMPLEMENTACIÓN

Metodología utilizada

[Pendiente de agregar — especificar si se siguió Scrum, Kanban, entregas por sprint u otro esquema formal; el historial de commits refleja desarrollo iterativo por rol de usuario y por ronda de actualización, pero no se documenta una metodología formal en el repositorio.]

Etapas del desarrollo

Con base en la cronología real del repositorio (control de versiones Git):

1. **Bootstrap inicial en Expo/React Native** (enero 2026) — descartado posteriormente.
2. **Migración e importación del frontend a Flutter** (`Initial Flutter frontend import from hearthy` , `feat: full hearthcoin app and backend integration`).
3. **Desarrollo iterativo por rol de usuario** — múltiples rondas identificadas en el historial como "Segunda/Tercera/Cuarta/Quinta/Sexta actualización", incluyendo dos hitos etiquetados como "Segunda Versión" / "Version 2.0".
4. **Políticas de privacidad y ajustes visuales** (retiro de modo oscuro, actualización de README).
5. **Integración con Original Pay** (conexión y pruebas de cobro con HC).
6. **Integración con Original Auth** (flujo de verificación de correo con códigos de 6 dígitos, integrado a la pantalla de registro, despliegue de funciones y migración de base de datos).
7. **Consolidación de versión** (`Heartcoin_final_v1`).
8. **Notificaciones y mapas** (implementación de notificaciones, múltiples iteraciones sobre el componente de mapa y los pines de ubicación).
9. **Preparación para publicación en Google Play** (actualización de firma de la aplicación).
10. **Rediseño de interfaz** (pantallas de login/registro, tab inferior, traducción de inglés a español, ajuste de gráficas del monitor de votaciones).
11. **Documentación técnica** (elaboración de Manual Técnico y demás documentos de soporte).
12. **Cierre de ronda de desarrollo** ("última actualización").

Periodo de mayor intensidad de trabajo: 20 al 31 de julio de 2026, precedido de trabajo disperso desde el periodo formal de estadías (06/mayo/2026).

Actividades realizadas

[Completar/ajustar según registro de bitácora propio de estadías; a partir de la evidencia técnica del repositorio se documentan las siguientes actividades concretas:]

- Migración completa del frontend de Expo/React Native a Flutter.
- Diseño e implementación de pantallas y servicios para los tres roles de usuario (Personal, Organización, Empresa).
- Diseño del modelo de datos en Supabase/Postgres, incluyendo políticas RLS por tabla.

- Versionado del esquema de base de datos mediante migraciones SQL (`SQL/migrations/001` a `006`).
- Implementación de funciones RPC `SECURITY DEFINER`: `checkin_iniciativa`, `redeem_beneficio`, `redeem_servicio`.
- Desarrollo de Edge Functions para integración con Original Auth, Original Pay y MediaAAS.
- Implementación de autenticación server-to-server mediante firma HMAC-SHA256 para las integraciones externas.
- Implementación de check-in por QR con validación de geolocalización.
- Implementación de mapa interactivo (Google Maps) para visualización de beneficios y servicios georreferenciados.
- Implementación de sistema de notificaciones basado en triggers de base de datos.
- Configuración de firma de la aplicación Android y cambio de `applicationId` para publicación en Google Play.
- Generación de cuentas y datos de demostración para pruebas funcionales.
- Elaboración de documentación técnica del proyecto (README, Manual Técnico, guías de integración, documentación de endpoints).

Configuración e instalación

Ver Manual Técnico (sección 6) para el detalle completo de requisitos, dependencias y procedimiento de instalación.

Desarrollo de funcionalidades

Detallado en la sección 4 (Solución implementada) y en el listado de pantallas/servicios de la sección 6.

Pruebas realizadas

- Pruebas funcionales de los flujos de check-in, votación, canje de beneficios/servicios y transacciones de HC utilizando las cuentas de demostración documentadas en `CUENTAS_DEMO.md`.
- Prueba de punta a punta del flujo de Original Pay (consulta de saldo, cobro y reembolso automático ante rechazo), confirmada como completada en `ORIGINAL_PAY_INTEGRATION.md`.
- Pruebas del flujo de verificación de correo con Original Auth, incluyendo el comportamiento en ambiente de prueba (`testToken`) y en ambiente productivo.
- Existen documentos adicionales de pruebas (`pruebas_y_validaciones.pdf`, `PRUEBAS_DESPLIEGUE_SEPARADO.pdf`) generados durante el proyecto cuyo contenido detallado no fue incorporado a este documento — se recomienda anexarlos íntegros (ver sección de Anexos).

Validaciones efectuadas

- Validación de políticas RLS por tabla (documentadas en `ENDPOINTS.md`).
- Validación de errores conocidos y controlados en las funciones RPC: `iniciativa_no_encontrada`, `metodo_no_es_qr`, `fuera_de_horario`, `fuera_de_rango`, `hc_insuficiente`, `ya_canjeado`, entre otros.
- Validación de idempotencia del reembolso automático en el webhook de Original Pay.

Ajustes implementados

- Corrección del tamaño de los pines en el mapa.
- Múltiples iteraciones sobre el componente de mapa hasta llegar a la versión vigente con Google Maps.
- Ajustes visuales en pantallas de login/registro y en la barra de navegación inferior.
- Traducción de la interfaz de inglés a español.
- Cambio de gráfica en el monitor de votaciones.
- Retiro del modo oscuro de la aplicación.

6. MANUAL TÉCNICO

Requisitos necesarios

- SDK de Dart `^3.11.5`.
- Flutter (versión compatible con el SDK de Dart anterior).
- Cuenta y proyecto activo de Supabase.
- Credenciales de acceso a los servicios Original Auth, Original Pay y MediaAAS (API Key + secreto para firma HMAC).
- Entornos de desarrollo Android/iOS configurados (Android Studio / Xcode según plataforma).

Dependencias (`pubspec.yaml`)

Paquete: `heartcoin` · **Versión de app:** `1.0.0+1` · **SDK Dart:** `^3.11.5`

Paquete	Versión
<code>supabase_flutter</code>	<code>^2.15.2</code>
<code>image_picker</code>	<code>^1.2.3</code>
<code>file_picker</code>	<code>8.1.2</code>
<code>geolocator</code>	<code>13.0.1</code>
<code>geocoding</code>	<code>^4.0.0</code>
<code>http</code>	<code>^1.6.0</code>
<code>mime</code>	<code>^2.0.0</code>
<code>http_parser</code>	<code>^4.1.2</code>
<code>shared_preferences</code>	<code>^2.5.5</code>
<code>qr_flutter</code>	<code>^4.1.0</code>
<code>mobile_scanner</code>	<code>^7.2.0</code>
<code>flutter_local_notifications</code>	<code>^22.1.0</code>
<code>google_maps_flutter</code>	<code>^2.18.0</code>
<code>google_maps_flutter_android</code>	<code>^2.19.12</code>
<code>google_maps_flutter_platform_interface</code>	<code>^2.16.0</code>
<code>cupertino_icons</code>	<code>^1.0.8</code>

Dev dependencies: `flutter_test` (SDK), `flutter_lints` (`^6.0.0`), `flutter_launcher_icons` (`^0.14.4`).

Configuración inicial

- Ícono de la app: `assets/logo_heartcoin_vista.png`, `min_sdk_android: 21`, fondo adaptativo `#FFFFFF` (configurado vía `flutter_launcher_icons`).
- Android `applicationId`: `com.theoriginallab.heartcoin` (`android/app/build.gradle.kts`).
- Credenciales de Supabase configuradas en `lib/main.dart`.

Procedimientos de instalación

[Pendiente de agregar — detallar comandos exactos utilizados para clonar, instalar dependencias (`flutter pub get`), levantar el entorno local y compilar para cada plataforma, así como el procedimiento específico de despliegue seguido, referenciando `manual_despliegue.pdf` y `DESPLIEGUE_INDEPENDIENTE.pdf` ya existentes en el repositorio.]

Parámetros importantes

- Base URL de Supabase: `https://bamrvwzpcqwoyubvomw.supabase.co` (proyecto `heartcoin-v1`).
- Las Edge Functions de Original Pay se desplegaron con `--no-verify-jwt`, ya que su seguridad se basa en la firma HMAC propia y no en el JWT de Supabase.
- Headers de autenticación HMAC utilizados en las integraciones externas: `X-API-Key`, `X-Timestamp`, `X-Nonce` (solo Original Auth), `X-Signature`.

Consideraciones técnicas

- Toda la lógica de negocio sensible (otorgamiento/descuento de HC, check-in, canjes) se ejecuta exclusivamente mediante funciones `SECURITY DEFINER` en Postgres, nunca desde el cliente directamente.
- El campo `description` de los cobros de Original Pay aún no se persiste en base de datos — pendiente de implementación.
- Existe una condición de carrera no atómica documentada en `redeem` (Original Pay) y en `redeem_beneficio` / `redeem_servicio`, sin límites de monto definidos actualmente.
- El webhook de Original Pay no cuenta con firma criptográfica real (limitación del lado de Original Pay, no de HeartCoin) — usa credenciales `token_app` / `secret_key` en el cuerpo de la solicitud.
- El campo `country` no se envía en el registro con Original Auth por incompatibilidad de formato (ISO alpha-2 vs. texto libre del formulario).

Solución de problemas comunes

Problema	Causa	Recomendación
Verificación de correo no llega en ambiente de prueba	El ambiente <code>test</code> de Original Auth no envía correos reales	Usar el <code>testToken</code> devuelto directamente por la API en ese ambiente
Error <code>FunctionException</code> no controlado al llamar Edge Functions	El SDK de Supabase requiere captura explícita de esta excepción	Envolver las llamadas a Edge Functions en manejo explícito de <code>FunctionException</code>

Problema	Causa	Recomendación
Discrepancia en conteos de datos demo	Múltiples documentos (<code>README.md</code> , <code>MANUAL_TECNICO.md</code> , <code>CUENTAS_DEMO.md</code>) reportan cifras distintas de cuentas/iniciativas/beneficios de prueba	Consultar directamente las tablas de Supabase (<code>personal_profiles</code> , <code>iniciativas</code> , <code>beneficios</code> , <code>servicios</code>) para obtener el conteo real vigente

Estructura del sistema

Ver diagrama de arquitectura y tabla de componentes en la sección 4.

Base de datos

Aproximadamente 20 tablas principales en Postgres (Supabase), todas con RLS habilitado:

Tabla	Propósito
<code>personal_profiles</code> / <code>organization_profiles</code> / <code>company_profiles</code>	Perfil según rol de usuario
<code>blocked_users</code>	Bloqueos entre usuarios
<code>publicaciones</code>	Publicaciones del feed
<code>post_likes</code> / <code>comments</code> / <code>comment_likes</code>	Interacciones sociales
<code>follows</code>	Relación de seguimiento
<code>notifications</code>	Notificaciones generadas por triggers
<code>iniciativas</code>	Iniciativas (Voluntariado / Crowdfunding / Social / Ahorro)
<code>iniciativa_votes</code>	Apoyos a iniciativas (no otorgan HC)
<code>iniciativa_checkins</code>	Registro de check-ins vía <code>checkin_iniciativa()</code>
<code>iniciativa_participants</code> / <code>iniciativa_join_requests</code>	Participación y solicitudes de unión a iniciativas
<code>proveedores</code>	Catálogo de proveedores para metas de ahorro
<code>votaciones</code>	Campañas de votación de organizaciones
<code>votacion_votes</code>	Votos (otorgan HC)
<code>votacion_comments</code>	Comentarios de votaciones
<code>hc_transactions</code>	Ledger único de movimientos de HC
<code>beneficios</code> / <code>beneficio_redemptions</code>	Beneficios de empresas y sus canjes
<code>servicios</code> / <code>servicio_redemptions</code>	Servicios de empresas y sus canjes

Tabla	Propósito
<code>saved_items</code>	Elementos guardados por el usuario

Migraciones versionadas (`SQL/migrations/`):

```
001_iniciativa_participantes.sql
002_beneficios_servicios_ubicacion.sql
003_publicaciones_iniciativa.sql
004_beneficios_destacado.sql
005_organization_profiles_lectura_publica.sql
006_servicios_destacado.sql
```

Documento complementario existente: `modelo_de_datos.pdf` (diagrama/detalle extendido del modelo de datos — anexo íntegro).

Módulos principales

Módulos por dominio funcional: Autenticación, Feed social, Iniciativas, Votaciones, Billetera/HC, Beneficios, Servicios, Mapa, Notificaciones, Certificados, Administración de cuenta.

APIs o servicios utilizados

HeartCoin no implementa un backend propio con rutas personalizadas; consume directamente los servicios autogenerados por Supabase:

1. **Auth** (`/auth/v1/*`) vía SDK `supabase_flutter` .
2. **PostgREST** (`/rest/v1/<tabla>`) — un endpoint por tabla, protegido por RLS.
3. **RPC** (`/rest/v1/rpc/<función>`): `checkin_iniciativa` , `redeem_beneficio` , `redeem_servicio` .
4. **Realtime** (WebSocket) sobre las tablas listadas en la sección 4.
5. **Edge Functions** (`/functions/v1/<nombre>`) — únicos endpoints verdaderamente personalizados:

Endpoint	Método	Función
<code>/functions/v1/upload-media</code>	POST	Proxy de subida de archivos a MediaAAS
<code>/functions/v1/delete-account</code>	POST	Eliminación de cuenta (Service Role Key)
<code>/functions/v1/originalauth-register</code>	POST	Registro contra Original Auth
<code>/functions/v1/originalauth-verify-email</code>	POST	Verificación de código de correo
<code>/functions/v1/originalauth-resend-verification</code>	POST	Reenvío de código de verificación
<code>/functions/v1/originalpay-balance</code>	GET	Consulta de saldo HC por correo
<code>/functions/v1/originalpay-redeem</code>	POST	Registro de cobro con HC

Endpoint	Método	Función
/functions/v1/originalpay-webhook	POST	Recepción de resultado de pago (con reembolso automático si es rechazado)

Flujo de información

Descrito en la sección 4 ("Flujo de funcionamiento").

7. RESULTADOS DEL PROYECTO

Resultados obtenidos

- Aplicación funcional en Flutter para Android e iOS, con los tres roles de usuario (Personal, Organización, Empresa) operativos de punta a punta.
- Integración completa y probada con Original Pay (consulta de saldo, cobro y reembolso automático).
- Integración funcional con Original Auth para verificación de correo en el registro.
- Modelo de datos versionado mediante migraciones SQL, sustituyendo el esquema aplicado manualmente en Supabase Studio.
- Aplicación configurada y firmada para publicación en Google Play (`applicationId` propio de The Original Lab).
- Conjunto de datos de demostración disponible para pruebas funcionales (cuentas Personal, Organización y Empresa con publicaciones, iniciativas, beneficios y servicios de ejemplo — ver `CUENTAS_DEMO.md` , documento con fecha más reciente entre las referencias existentes; se reconoce la existencia de discrepancias menores en los conteos exactos respecto a versiones previas de `README.md` y `MANUAL_TECNICO.md` , ya identificadas internamente en la documentación del propio proyecto).

Mejoras implementadas

- Migración de la base tecnológica original (Expo/React Native) a Flutter, unificando el desarrollo multiplataforma.
- Rediseño de pantallas clave (login, registro, navegación inferior) y traducción completa de la interfaz al español.
- Iteración del componente de mapa hasta una versión estable basada en Google Maps con pines corregidos.
- Retiro de funcionalidad de modo oscuro para simplificar el mantenimiento visual.

Comparativa antes/después

Aspecto	Antes	Después
Tecnología base	Expo / React Native	Flutter (Android + iOS)
Esquema de base de datos	Cambios aplicados manualmente en Supabase Studio	Migraciones SQL versionadas (<code>SQL/migrations/001 – 006</code>)
Identidad de usuario	Sin verificación de correo real	Verificación de correo mediante Original Auth
Pagos/canjes con terceros	Sin integración externa	Integración funcional con Original Pay (consulta de saldo y cobros)
Identificador de aplicación	<code>com.example.heartcoin</code>	<code>com.theoriginallab.heartcoin</code> , firmada para Google Play
Idioma de interfaz	Inglés	Español

Beneficios para la empresa

- The Original Lab cuenta con un producto propio, funcional y de marca (`com.theoriginallab.heartcoin`), que además sirve como validación en producción de Original Auth y Original Pay.
- Arquitectura sin backend propio que mantener (Supabase + Edge Functions), reduciendo el costo operativo de infraestructura.

Indicadores o métricas disponibles

[Pendiente de agregar — no se identificaron en el repositorio métricas de uso real en producción (usuarios activos, transacciones de HC reales, tasa de canje, etc.). Los datos actualmente disponibles corresponden únicamente a cuentas y contenido de demostración para pruebas funcionales, documentados en `CUENTAS_DEMO.md` .]

8. OPERACIÓN Y MANTENIMIENTO

Recomendaciones de uso

- Consultar siempre las tablas de Supabase directamente para obtener conteos actuales de usuarios, iniciativas, beneficios y servicios, dado que los documentos estáticos del proyecto (`README.md` , `MANUAL_TECNICO.md` , `CUENTAS_DEMO.md`) han mostrado desactualizarse entre sí.
- Mantener actualizada la carpeta `SQL/migrations/` como único mecanismo de cambio de esquema, evitando modificaciones manuales directas en Supabase Studio.

Buenas prácticas

- Utilizar exclusivamente las funciones `SECURITY DEFINER` (`checkin_iniciativa` , `redeem_beneficio` , `redeem_servicio`) para cualquier operación que afecte el saldo de HC o el estado de participación, nunca escritura directa desde el cliente.
- Mantener la autenticación HMAC-SHA256 como mecanismo de seguridad server-to-server para toda integración externa nueva, siguiendo el patrón ya implementado en `_shared/originalauth.ts` .

Mantenimiento preventivo

- Revisar periódicamente la vigencia de las versiones de dependencias listadas en `pubspec.yaml` , particularmente los paquetes relacionados con Google Maps y geolocalización, sujetos a actualizaciones frecuentes de las plataformas Android/iOS.
- Monitorear el comportamiento del webhook de Original Pay ante la ausencia de firma criptográfica real de ese lado, dado que representa una superficie de riesgo fuera del control directo de HeartCoin.

Posibles mejoras futuras

- Implementar persistencia del campo `description` en los cobros registrados por Original Pay.
- Resolver la condición de carrera no atómica identificada en las funciones de canje (`redeem_beneficio` , `redeem_servicio` , `redeem` de Original Pay), idealmente mediante bloqueos transaccionales o control de concurrencia optimista.
- Definir límites de monto para los cobros de Original Pay, actualmente sin restricción documentada.
- Resolver el envío del campo `country` a Original Auth, adaptando el formulario de registro a formato ISO alpha-2 o normalizando el valor antes del envío.
- Consolidar en un único documento vivo (por ejemplo, este mismo) los conteos de datos de demostración, evitando mantener múltiples archivos con información divergente.

Riesgos o consideraciones

- Ausencia de firma criptográfica real en el webhook de `originalpay-webhook`, dependiente de una mejora en la plataforma Original Pay.
- Falta de límites de monto en los cobros con HC, lo que podría representar un riesgo operativo si se habilita tráfico real sin control adicional.
- Documentación histórica (README/Manual Técnico) con referencias desactualizadas (por ejemplo, tecnología de mapas) que deben depurarse para evitar confusión en el equipo técnico.

9. CONCLUSIONES

El proyecto HeartCoin cumplió el objetivo planteado: entregar una aplicación móvil funcional en Flutter para Android e iOS que integra participación social, un sistema propio de moneda de intercambio (HeartCoin) y dos servicios estratégicos de The Original Lab — Original Auth y Original Pay — validándolos en un producto real y operativo. La migración desde una base tecnológica inicial en Expo/React Native hacia Flutter, junto con la introducción de migraciones SQL versionadas, refleja una maduración técnica del proyecto durante el periodo de desarrollo.

El valor aportado a la empresa se concentra en dos frentes: por un lado, un producto propio con identidad de marca (`com.theoriginallab.heartcoin`) listo para publicación en Google Play; por otro, la validación funcional de extremo a extremo de Original Pay (incluyendo manejo de rechazos y reembolsos automáticos) y de Original Auth, sirviendo como referencia técnica para futuras integraciones de terceros con ambas plataformas.

A nivel técnico y operativo, el proyecto deja como base una arquitectura de bajo costo de mantenimiento (sin backend propio, apoyada íntegramente en Supabase), con seguridad a nivel de fila en toda la base de datos y funciones de escritura sensible debidamente aisladas mediante `SECURITY DEFINER` . Quedan pendientes, y documentadas explícitamente en este entregable, mejoras concretas de robustez (persistencia de campos, control de concurrencia, límites de monto, firma del webhook) que se recomienda priorizar antes de una eventual salida a producción con usuarios reales.

ANEXOS

Se sugiere complementar este documento con los siguientes anexos, ya existentes como archivos independientes en el repositorio del proyecto:

Documentos técnicos existentes (anexar íntegros)

Documento	Contenido sugerido
arquitectura_tecnica.pdf	Detalle extendido de arquitectura
modelo_de_datos.pdf	Diagrama y detalle completo del modelo de datos
apis_heartcoin.pdf	Detalle de APIs
flujos_transaccionales.pdf	Diagramas de flujo de transacciones de HC
manual_despliegue.pdf / DESPLIEGUE_INDEPENDIENTE.pdf	Procedimiento de despliegue
pruebas_y_validaciones.pdf / PRUEBAS_DESPLIEGUE_SEPARADO.pdf	Evidencia detallada de pruebas
DISEÑO_FLUJOS_USUARIO.pdf	Diagramas de flujo de usuario (UX)
estructura_implementacion_frontend.pdf / separacion_frontend_backend.pdf	Detalle de implementación frontend/backend
informe_final_funcionalidades.pdf / INFORME_FUNCIONALIDADES_IMPLEMENTADAS.pdf	Detalle de funcionalidades entregadas
analisis_integracion_heartcoin_iaas.pdf	Análisis de integración con infraestructura
informe-de-analisis-de-arquitectura-y-evaluacion-tecnica-heartcoin.pdf	Evaluación técnica de arquitectura
heartcoin_analisis_funciones_pendientes.pdf	Funciones pendientes identificadas
integracion_flujos_validaciones_frontend.pdf	Validaciones de integración frontend

Otras evidencias sugeridas

- Capturas de pantalla de la aplicación por rol de usuario (Personal, Organización, Empresa). *[Pendiente de agregar]*
- Evidencia visual del panel de Supabase (tablas, políticas RLS, Edge Functions desplegadas). *[Pendiente de agregar]*
- Tabla de cuentas de demostración completa (CUENTAS_DEMO.md) para referencia de pruebas.
- Guías de integración de referencia: ORIGINAL_AUTH_INTEGRATION.md , ORIGINAL_PAY_INTEGRATION.md , guia_auth_apps_externas.md , guia_integracion_auth_mobile.md , mediaas-client-api-guide.md .

- Código relevante de referencia: funciones `_shared/originalauth.ts`, `checkin_iniciativa`, `redeem_beneficio`, `redeem_servicio`. *[Incluir snippets si se requiere en la versión final impresa/entregada]*

Fin del documento.